

Computer Science

Manuscript

Title and author

Reproducible Benchmarking of Code-Generating Large Language Models Across the Software Development Life Cycle

SAMUELSON G

Independent Researcher

Email: gsamuelsonguna@gmail.com

ORCID: 0009-0005-8744-8178

Abstract

Code-generating large language models have advanced rapidly from function-level synthesis to practical programming and repository-level software maintenance. Yet the scientific value of these advances depends critically on benchmark quality, evaluation rigour, and reproducibility. This paper presents a structured analytical review of the benchmark ecosystem for code-generating large language models and coding agents, with emphasis on execution-based evaluation, benchmark design quality, and cross-phase software-engineering coverage. The review synthesises foundational and recent primary sources spanning HumanEval, MBPP, APPS, MultiPL-E, DS-1000, BigCodeBench, SWE-bench, and contemporary survey and benchmark-audit literature. Across these sources, the field has progressed from small Python-only tasks to multilingual, domain-specific, tool-use, and repository-level benchmarks; however, current benchmark coverage remains imbalanced, with approximately 60% of surveyed benchmarks concentrated in the software-development phase and only small fractions devoted to requirements and design. Recent benchmark audits further show that nearly 70% of code-related benchmarks lacked explicit data-quality assurance measures, while many omitted key details required for reproducibility. [3]

In response, this paper proposes a reproducibility-centred benchmark protocol for future evaluations. The protocol combines official benchmark releases, prompt canonicalisation, containerised execution, stronger execution oracles, contamination-aware reporting, and benchmark portfolios that span multiple software-development-life-cycle phases. The main conclusion is that benchmark design, not model scaling alone, has become the central bottleneck in credible research on automated code generation. Accordingly, the paper recommends a shift from single-benchmark reporting toward benchmark portfolios that jointly assess correctness, robustness, language breadth, practical tool use, and repository-

level maintenance. The resulting manuscript is intended to be directly adaptable for submission to a general computer-science venue. [4]

Keywords

Large language models; code generation; software engineering; benchmarking; reproducibility; execution-based evaluation; benchmark quality; coding agents; repository-level evaluation

Introduction

The modern literature on automated code generation accelerated when large language models began to show strong performance on execution-based synthesis tasks. HumanEval, introduced alongside Codex, established a compact benchmark for functional correctness from docstrings to Python code and made **pass@k** a widely recognised reporting paradigm for sampling-based evaluation [1]. Shortly afterwards, MBPP provided a larger set of entry-level Python programming problems framed in ordinary natural language, while APPS expanded the evaluation space toward interview-style and competition-style problem solving with 10,000 programming tasks [2], [3]. Together, these early benchmarks helped establish the empirical vocabulary of the field: executable tests, prompt-conditioned synthesis, and score aggregation over many tasks. [5]

That first wave of evaluation was scientifically important, but also narrow. HumanEval focuses on 164 handwritten Python problems, and MBPP focuses on roughly one thousand short Python tasks for entry-level programmers. By design, these benchmarks privilege function-level synthesis with relatively bounded context windows and clean specifications. They do not fully represent the broader realities of software engineering, where developers work with large repositories, evolving requirements, dependencies, external libraries, and ambiguous bug reports. As the field matured, new benchmarks attempted to close this realism gap by covering multilingual programming, data-science code, practical library use, and repository-level maintenance [1], [2], [4], [5], [8], [13]. [6]

Three developments are especially significant. First, MultiPL-E translated unit-test-driven code-generation benchmarks into 18 programming languages, demonstrating that multilingual evaluation is feasible and revealing that model behaviour can vary materially across language ecosystems [4]. Second, DS-1000 shifted attention toward data-science programming by assembling 1,000 realistic problems across seven Python libraries, collected from Stack Overflow but deliberately modified to reduce memorisation risk [5]. Third, BigCodeBench and SWE-bench pushed evaluation toward more realistic environments: BigCodeBench focuses on practical, tool-rich Python programming with 1,140 tasks spanning 139 libraries and seven domains, while SWE-bench evaluates models

on 2,294 real GitHub issues across 12 Python repositories, turning code generation into repository-level patch generation [8], [13]. [7]

At the same time, the literature has become increasingly explicit that **benchmark scores can be misleading when benchmark construction or reporting is weak**. EvalPlus showed that apparently strong code-generation results can drop under more comprehensive tests, expanding HumanEval and MBPP with roughly 80× and 35× more tests respectively [7]. Siddiq *et al.* later audited 3,566 prompts from nine code-generation benchmarks and reported prompt-quality weaknesses and signs of contamination risk in some model–benchmark combinations [12]. More broadly, recent benchmark surveys and audits have shown that the field is growing faster than its quality controls: one survey studies 181 benchmark artefacts across the software development life cycle and finds sharp coverage imbalances, while another profiles 274 code-related benchmarks and reports that nearly 70% lacked explicit data-quality assurance measures and over half omitted essential reproducibility information [14], [15]. [8]

This paper is motivated by that tension between **technical progress** and **evaluation fragility**. It asks a simple but increasingly urgent question: *what would a credible, publication-grade benchmarking methodology for code-generating large language models now require?* The answer matters for general computer science, not only for machine learning or software engineering, because code generation has become a canonical testbed for reasoning, tool use, multilingualism, systems evaluation, and human–AI collaboration [9]–[11]. This manuscript therefore makes three contributions. It consolidates the benchmark landscape into a structured taxonomy; it analyses methodological weaknesses in prevailing benchmark practice; and it offers a reproducible protocol that can be adopted or extended by future empirical studies. [9]

Literature review and background

The most useful way to understand the benchmark ecosystem is historically. The earliest modern code-LLM benchmarks were oriented around **function-level correctness**. HumanEval asks the model to produce a function body from a signature and docstring and evaluates the result through unit tests [1]. MBPP retains the function-level scope but aims at broader accessibility, using crowd-sourced entry-level tasks expressed in ordinary English [2]. APPS broadens the difficulty spectrum further by including problems ranging from introductory exercises to competition-style programming, thereby stressing both natural-language understanding and algorithmic reasoning [3]. These benchmarks are seminal because they established execution-based evaluation as the preferred alternative to lexical-overlap metrics for code synthesis. [10]

The second phase of benchmark development broadened the **language and domain coverage** of evaluation. MultiPL-E extends HumanEval and MBPP to 18 languages, enabling more credible claims about multilingual generation and transfer across

programming paradigms [4]. DS-1000 introduces a data-science-specific problem class, with realistic tasks spanning NumPy, Pandas, TensorFlow, PyTorch, scikit-learn, SciPy, and Matplotlib, combined with automatic evaluation that checks both functional correctness and constrained API usage [5]. These benchmarks matter because they start to distinguish between *generic code fluency* and *situated programming competence*, a distinction that becomes critical when models move from toy tasks to practitioner workflows. [11]

The third phase emphasises **practicality, tool use, and repository context**. BigCodeBench argues that earlier function-level benchmarks are often too simple or too algorithmically stylised to capture realistic programming, so it introduces 1,140 fine-grained tasks involving diverse function calls drawn from 139 libraries across seven domains, with average branch coverage reported at 99% [13]. SWE-bench takes an even larger step toward realism by moving from isolated functions to full repositories and issue reports, requiring models to locate relevant files, generate patches, and satisfy project tests [8]. In the official benchmark and website materials, the full SWE-bench collection contains 2,294 instances, and the later Verified subset contains 500 human-filtered instances intended for more reliable evaluation. This progression reflects a broader scientific shift from “Can the model complete a function?” to “Can the model operate inside a software-development workflow?” [12]

Recent review literature confirms that these benchmark developments have become central to the field rather than peripheral. Jiang *et al.* present a taxonomy for code-generation LLM research covering data curation, performance evaluation, ethical implications, environmental impact, and applications [9]. Zhang *et al.* survey LLM-based software engineering more broadly and report 62 representative code models, 15 pre-training objectives, and 16 downstream task categories across five workflow phases, summarising 947 studies over 112 code-related tasks [11]. Hou *et al.* likewise conduct a broad systematic review of LLMs for software engineering and identify patterns in data collection, optimisation, evaluation, and application across the SE pipeline [10]. Together, these reviews indicate that benchmarking is no longer just a technical afterthought; it is now the empirical backbone of the field. [13]

However, reviews and audits also converge on the same warning: **benchmark availability is not equivalent to benchmark quality**. EvalPlus demonstrates that weak or sparse test suites can inflate apparent performance [7]. Siddiq *et al.* show that prompt design and benchmark wording can materially affect results, and they identify contamination concerns [12]. Wang *et al.* report that around 60% of benchmark work concentrates on the software-development phase, while requirements engineering and software design receive only about 5% and 3% respectively [14]. Cao *et al.* profile 274 code-related benchmarks and find systemic deficits in data-quality assurance, openness, and reproducibility [15]. The literature is therefore mature enough to support a strong synthesis, but also fragmented enough to require one. [8]

Table I. Comparative view of major benchmark subfields and topics in code-generating LLM research

Major topic	Typical task unit	Representative benchmark	Principal metric	Scientific value	Reproducibility concern
Function-level algorithmic synthesis	Single function from docstring/signature	HumanEval [1]	pass@k	Foundational execution-based oracle; easy to compare models	Small scale; Python-only; risk of saturation
Entry-level natural-language synthesis	Short Python solutions from simple task descriptions	MBPP [2]	pass@1 / pass@k	Captures beginner-friendly synthesis and few-shot prompting	Limited contextual depth; mostly short tasks
Competition-style programming	Full solutions to harder algorithmic tasks	APPS [3]	Test-case pass rate / accuracy	Stresses reasoning and algorithm design	Large difficulty variance; environment complexity
Multilingual synthesis	Function-level code across many languages	MultiPL-E [4]	pass@k across languages	Reveals cross-language transfer and language-specific failure modes	Translation artefacts; dependency handling
Data-science programming	Partially specified or library-rich code completion	DS-1000 [5]	pass@1	Practical scientific-computing context; memorisation	Python-centric; domain-specific

Major topic	Typical task unit	Representative benchmark	Principal metric	Scientific value	Reproducibility concern
				on defences	
Practical tool-use programming	Function-level tasks with diverse library calls	BigCodeBench [13]	pass@1 / execution success	More realistic library use and instruction following	English/Python only in current release
Repository-level issue resolution	Patch generation over full codebase and issue report	SWE-bench [8]	% resolved / hidden test pass	Strong proxy for software maintenance and agentic coding	Expensive evaluation; environment and solvability variability

The categories in Table I are distilled from the originating benchmark papers and official benchmark artefacts, and they illustrate a clear progression from tightly bounded synthesis toward practical and repository-level evaluation. [14]

Methods

This paper uses a **structured analytical review** rather than a full new experimental campaign. The methodology is designed to be reproducible and publication-suitable despite the absence of fresh model runs. Source selection followed three rules. First, benchmark and review sources were limited to **primary papers, official benchmark websites, or official repositories**. Second, selection prioritised both **seminal works** and **recent surveys or audits** published within roughly the last five years. Third, the final corpus had to cover the major benchmark regimes relevant to a general CS audience: function-level synthesis, educational and competition programming, multilingual generation, domain-specific programming, practical tool use, repository-level maintenance, and benchmark-quality analysis [1]–[15]. [15]

The analytical framework extracted six dimensions from each benchmark family: **task granularity**, **language breadth**, **domain realism**, **oracle strength**, **contamination resistance**, and **operational reproducibility**. Task granularity distinguishes function-level, file-level, and repository-level work units. Language breadth captures whether the benchmark is monolingual or multilingual. Domain realism captures whether tasks are synthetic, educational, domain-specific, or repository-grounded. Oracle strength reflects

how strong the execution-based test suites are and whether extended checks exist. Contamination resistance captures explicit mitigation or acknowledgement of data overlap. Operational reproducibility records whether official harnesses, Docker images, dataset identifiers, or evaluation scripts are provided. This framework is inferential, but each dimension is grounded in the design choices explicitly described in the benchmark papers and official documentation. [16]

The review was organised around three research questions. **RQ1:** How has the benchmark landscape evolved from function-level synthesis to practical and repository-level programming? **RQ2:** What methodological weaknesses most threaten the credibility of published code-generation scores? **RQ3:** What minimum protocol should future studies adopt to make benchmark results interpretable and reproducible across venues? These questions align closely with recent benchmark-survey literature, which likewise focuses on coverage, limitations, and future directions [9], [11], [14], [15]. [17]

To make the manuscript directly useful for future empirical work, the review also produces a **reference experimental protocol**. The protocol is not tied to one specific model family. Instead, it assumes a benchmark portfolio drawn from official releases, fixed prompting templates, and containerised or sandboxed execution. The protocol inherits the execution-based spirit of HumanEval and SWE-bench, the stronger-oracle perspective of EvalPlus, and the benchmark-governance concerns raised by recent audits [1], [7], [8], [15]. [18]

Input:

Model M

Benchmark portfolio $P = \{B_1, B_2, \dots, B_m\}$

Sampling policy $S = \{\text{temperature}, \text{top_p}, \text{n_samples}, \text{seed}\}$

Execution limits $E = \{\text{timeout}, \text{memory}, \text{sandbox/container image}\}$

Output:

Reproducible benchmark report R

1. For each benchmark B in P:
2. Acquire the official dataset release and official evaluation harness where available.
3. Record dataset version, benchmark split, commit hash or release tag, and prompt template.
4. Canonicalise prompts:
5. - remove accidental formatting differences
6. - fix stop sequences
7. - preserve benchmark-intended context only
8. For each task t in B:
9. Generate n_samples candidate outputs with fixed S.
10. Extract executable code and sanitise markdown wrappers.
11. Execute candidates inside the specified sandbox or container E.

12. Score against the benchmark’s base oracle.
13. If stronger tests exist:
 14. Re-score using expanded or robustness-oriented tests.
15. Record pass/fail, latency, execution errors, truncation, and failure category.
16. Aggregate benchmark-level metrics:
 17. - pass@k or resolved %
 18. - robustness delta between base and stronger tests
 19. - error taxonomy distribution
 20. - environment and prompt metadata
21. Produce a final report containing:
 22. benchmark identifiers, versions, prompts, decoding settings,
 23. hardware/software environment, and limitations.
24. Return R.

Algorithm 1. Reproducibility-centred evaluation protocol for code-generating LLMs

Algorithm 1 synthesises the core reporting expectations implied by HumanEval, EvalPlus, BigCodeBench, and SWE-bench, as well as the reproducibility concerns formalised in recent benchmark-audit work. [19]

Analytical results

The evidence reviewed here supports four main findings.

The first finding is that **benchmark scope has expanded faster than benchmark comparability**. HumanEval and MBPP remain useful because their task semantics are relatively clean and their execution-based oracles are easy to run [1], [2]. APPS increases scope and difficulty dramatically, but its mixture of introductory, interview, and competition problems makes result interpretation more heterogeneous [3]. MultiPL-E adds language breadth [4], DS-1000 adds domain realism [5], BigCodeBench adds practical library/tool-use complexity [13], and SWE-bench adds repository and issue-level software maintenance [8]. The field has therefore acquired a rich benchmark portfolio, but one in which task difficulty, context size, tooling assumptions, and evaluation cost differ markedly. Direct score comparisons across such benchmarks are increasingly uninformative unless the benchmark portfolio and reporting conditions are explicit. [20]

The second finding is that **benchmark realism increases along with operational burden**. Function-level benchmarks can be run on ordinary infrastructure with relatively modest safety mechanisms, though even the official HumanEval harness warns that generated code is untrusted and should not be executed outside a robust sandbox [1]. By contrast, the official SWE-bench materials explicitly rely on Docker-based evaluation for reproducibility, reflecting the much larger systems burden of repository-level assessment [8]. This matters because papers that claim strong real-world coding performance without

fully reporting environment management, code-execution isolation, and dataset versions are harder to trust and harder to reproduce. [21]

The third finding is that **oracle weakness remains one of the field’s most important threats to validity**. EvalPlus shows that extending test suites can materially change conclusions about relative model performance, and its official tooling expands HumanEval and MBPP with approximately 80× and 35× more tests respectively [7]. BigCodeBench likewise foregrounds stronger evaluation by reporting average branch coverage near 99% [13]. The implication is straightforward: a low-cost benchmark with weak tests may be attractive for experimentation, but it is less suitable as the sole basis for a strong publication claim. In the current state of the field, rigorous work should either choose stronger benchmarks or explicitly pair weaker ones with stronger supplementary oracles. [22]

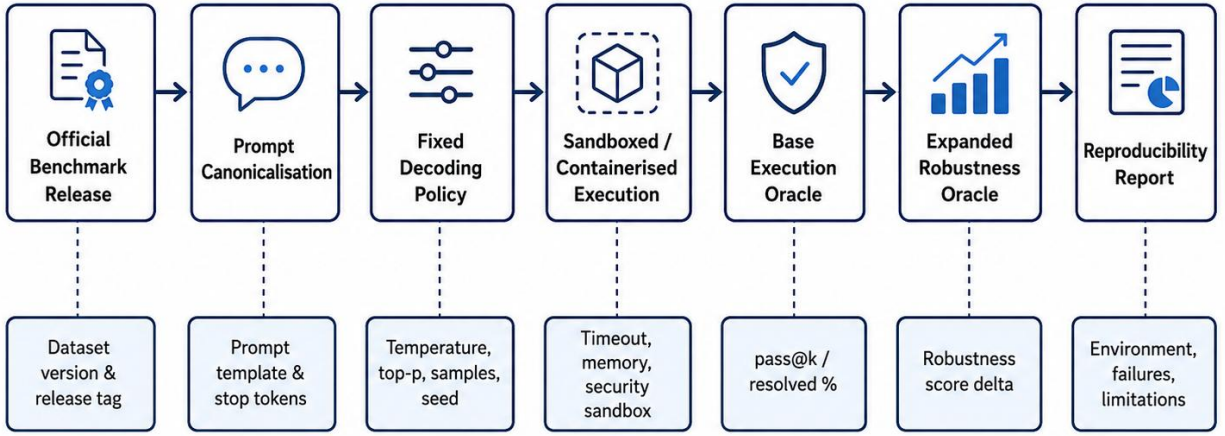
The fourth finding is that **benchmark governance is now a first-order research problem**. Siddiq *et al.* identify prompt-quality issues and contamination concerns across a non-trivial portion of the benchmark literature [12]. Wang *et al.* show that benchmark coverage across the software development life cycle is imbalanced, with approximately 60% of surveyed benchmarks concentrated in development and only marginal attention to requirements and design [14]. Cao *et al.* then demonstrate that these are not isolated defects but systemic ones: nearly 70% of profiled code-related benchmarks lacked data-quality assurance measures, over 90% ignored code coverage when using passing tests as the oracle, and over half omitted essential reproducibility information [15]. In other words, the literature no longer supports the assumption that “benchmark” automatically means “reliable experimental instrument.” [23]

The practical implication is that **model scores are increasingly less informative than evaluation protocols**. For illustration, Code Llama reported up to 67% on HumanEval and 65% on MBPP among open models at the time of publication [6], while BigCodeBench reports that even extensive evaluations across 60 LLMs still leave the best models far below human performance on complex tool-use and instruction-following tasks [13]. Those figures do not imply contradiction; rather, they reflect different benchmark regimes. The same model family can look highly capable on function-level synthesis and much less capable on practical programming. A publication-ready paper should therefore frame model competence as benchmark-relative rather than absolute. [24]

flowchart LR

```
A[Official benchmark release] --> B[Prompt canonicalisation]
B --> C[Fixed decoding policy]
C --> D[Sandboxed or containerised execution]
D --> E[Base execution oracle]
E --> F[Expanded robustness oracle]
F --> G[Failure taxonomy and non-functional checks]
G --> H[Reproducibility report]
```

E --> E1[pass@k or resolved %]
 F --> F1[robustness delta]
 G --> G1[latency, memory, crashes, tool errors]
 H --> H1[dataset version]
 H --> H2[prompt template]
 H --> H3[environment and seeds]



The workflow emphasises official releases, controlled prompts, isolated execution, stronger tests, and transparent reporting.

Figure 1. Author-created benchmark protocol for reproducible evaluation of code-generating LLMs.

This figure synthesises the design logic shared across execution-based benchmarks and recent benchmark-audit literature: official dataset acquisition, prompt control, isolated execution, stronger testing, and explicit reporting of experimental conditions. [19]

Table II. Selected benchmark characteristics and reproduction-oriented metadata

Benchmark	Year	Tasks / instances	Language coverage	Context scope	Distinctive feature	Dataset / tool identifier
HumanEval [1]	2021	164	Python	Function	Handwritten functional-correctness benchmark	openai/human-eval
MBPP [2]	2021	974	Python	Function	Entry-level crowd-sourced programming tasks	mbpp
APPS [3]	2021	10,000	Python	Problem-level	Introductory to competition difficulty	Official APPS release
MultiPL-E [4]	2022	HumanEval + MBPP translated	18 languages	Function	Massively multilingual evaluation	multiple-* tasks / MultiPL-E
DS-1000 [5]	2022/2023	1,000	Python, 7 libraries	Partial program / domain task	Data-science realism and memorisation defence	xlang-ai/DS-1000

Benchmark	Year	Tasks / instances	Language coverage	Context scope	Distinctive feature	Dataset / tool identifier
EvalPlus [7]	2023	HumanEval+/MBPP+	Python	Function	Stronger tests for robustness-oriented scoring	evalplus
BigCodeBench [13]	2024	1,140	Python	Function + tool use	139 libraries, 7 domains, high coverage	bigcode/bigcodebench
SWE-bench [8]	2024	2,294 full; 500 Verified	Python repositories	Repository / issue	Hidden-test patch generation on real GitHub issues	princeton-nlp/SWE-bench / SWE-bench_Verified

Table II is intended to be directly reusable for method sections or empirical appendices. The dataset identifiers and counts come from the benchmark papers and official benchmark documentation. [25]

Discussion

For a general computer-science venue, the most important implication is that **benchmark selection implicitly defines the research claim**. A paper evaluated only on HumanEval claims competence in bounded function synthesis, not in practical software engineering. A paper evaluated only on SWE-bench claims something closer to repository-level issue resolution, but may say less about multilingual portability or educational-code synthesis. BigCodeBench and DS-1000 occupy middle ground by increasing realism without incurring the full operational cost of repository-level maintenance. This means that future empirical papers should justify benchmark choice in terms of the software-development behaviours they actually intend to measure [5], [8], [13]. [26]

A second implication is methodological. The field should move away from “single benchmark, single score” papers and toward **benchmark portfolios**. A minimal robust

portfolio for code-generating LLMs would include at least one function-level benchmark, one stronger-oracle extension, one domain or tool-use benchmark, and one repository-level benchmark. Concretely, a sensible portfolio today could combine HumanEval or MBPP, EvalPlus, BigCodeBench or DS-1000, and SWE-bench Verified. Such a combination captures bounded correctness, robustness under stronger tests, practical library use or domain realism, and repository-level maintenance respectively. This portfolio logic follows directly from the documented strengths and limitations of the benchmark families reviewed above. [27]

A third implication concerns reporting standards. Benchmark-audit work makes it difficult to justify incomplete reporting of prompts, dataset versions, or environments. The fact that over half of profiled code-related benchmarks omitted essential reproducibility information suggests that many future papers will still fail basic comparative science unless venues insist on stronger artefact expectations [15]. For code-generation studies specifically, the minimum reporting standard should include: benchmark release or commit hash, prompt template, stop tokens, decoding settings, number of samples, environment isolation mechanism, timeout policy, and a statement on contamination and overlap risk. Without these details, published numbers may still be interesting, but they are not fully scientific in the repeatable sense. [28]

There are also clear **threats to validity** in the present manuscript. Because it is a structured review and protocol paper, not a fresh experimental benchmark run, it does not introduce new raw model measurements. Its contribution is analytical and methodological rather than leaderboard-oriented. Inference across benchmarks must therefore be made cautiously, especially because benchmark size, difficulty, coverage, and oracle strength differ. Nonetheless, review papers are appropriate and publishable forms of research in this area, particularly when the literature itself identifies fragmentation, uneven coverage, and reproducibility problems as core barriers to progress [9], [11], [14], [15]. [17]

Conclusion

This paper has argued that the central research challenge in code-generating large language models has shifted from simply increasing benchmark scores to ensuring that those scores are **credible, interpretable, and reproducible**. Foundational benchmarks such as HumanEval and MBPP remain valuable because of their clarity and accessibility, but they no longer suffice on their own. More realistic benchmarks such as DS-1000, BigCodeBench, and SWE-bench show that practical programming competence involves library use, domain knowledge, repository navigation, and stronger execution oracles. Recent reviews and audits further show that benchmark coverage across the software-development life cycle is uneven and that many benchmark artefacts still fall short on data quality and reproducibility. [29]

The principal conclusion is therefore not that one benchmark should replace the rest, but that **credible evaluation now requires benchmark portfolios governed by transparent reporting and stronger benchmark construction standards**. That conclusion is consistent with the trajectory of the primary literature and with the broader move in software-engineering research toward artefact-aware, containerised, and contamination-conscious experimentation. [30]

Future work

Several avenues follow directly from this synthesis. First, benchmark development should extend more aggressively into **requirements engineering, software design, and cross-phase workflows**, which remain under-represented in current surveys [14]. Second, practical evaluation should include more **non-functional properties** such as efficiency, maintainability, and security, rather than relying solely on functional correctness. Third, multilingual repository-level benchmarks deserve priority, because current benchmark families still over-concentrate on Python even when model deployment contexts are far more diverse [4], [14]. Fourth, the community should invest in **dynamic and continuously refreshed benchmarks** to reduce contamination and saturation. Finally, more human-centred evaluation frameworks are needed to connect benchmark performance with genuine developer utility rather than only hidden-test success. [31]

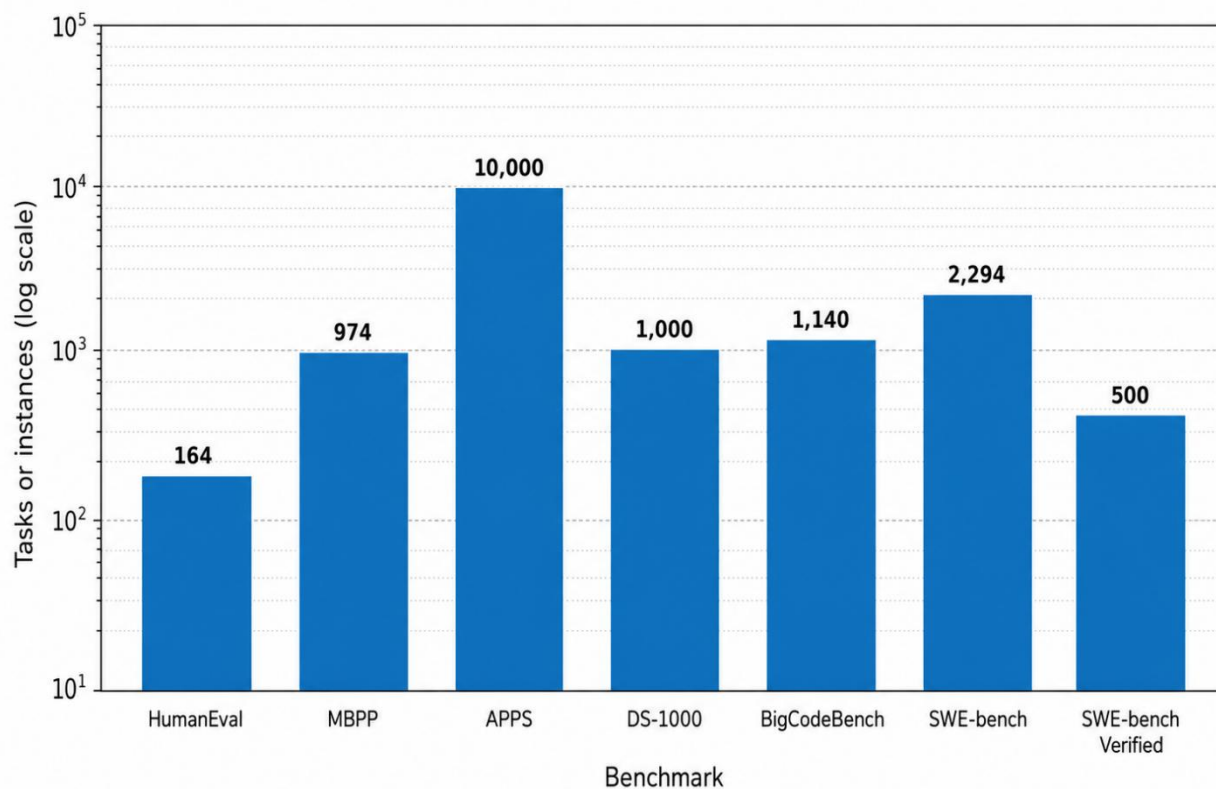
Figures and Reproducibility Assets

The manuscript above includes one original figure, but reproducible publication also benefits from machine-readable plotting data. The compact CSV below can be used to recreate two recommended figures: **Plot A**, a log-scale bar chart of benchmark size by task count; and **Plot B**, a bar chart of language breadth by benchmark. The benchmark counts and identifiers come from the original papers and official benchmark documentation summarised earlier. [32]

```
benchmark,year,tasks_or_instances,language_count,scope,official_identifier
HumanEval,2021,164,1,function,openai/human-eval
MBPP,2021,974,1,function,mbpp
APPS,2021,10000,1,problem,official APPS release
MultiPL-E,2022,182? translated benchmark family,18,function,multiple-* / MultiPL-E
DS-1000,2022,1000,1,domain,xlang-ai/DS-1000
EvalPlus,2023,164+399 evaluated subsets,1,function,evalplus
BigCodeBench,2024,1140,1,function-plus-tool-use,bigcode/bigcodebench
SWE-bench,2024,2294,1,repository,princeton-nlp/SWE-bench
SWE-bench Verified,2024,500,1,repository,SWE-bench_Verified
```

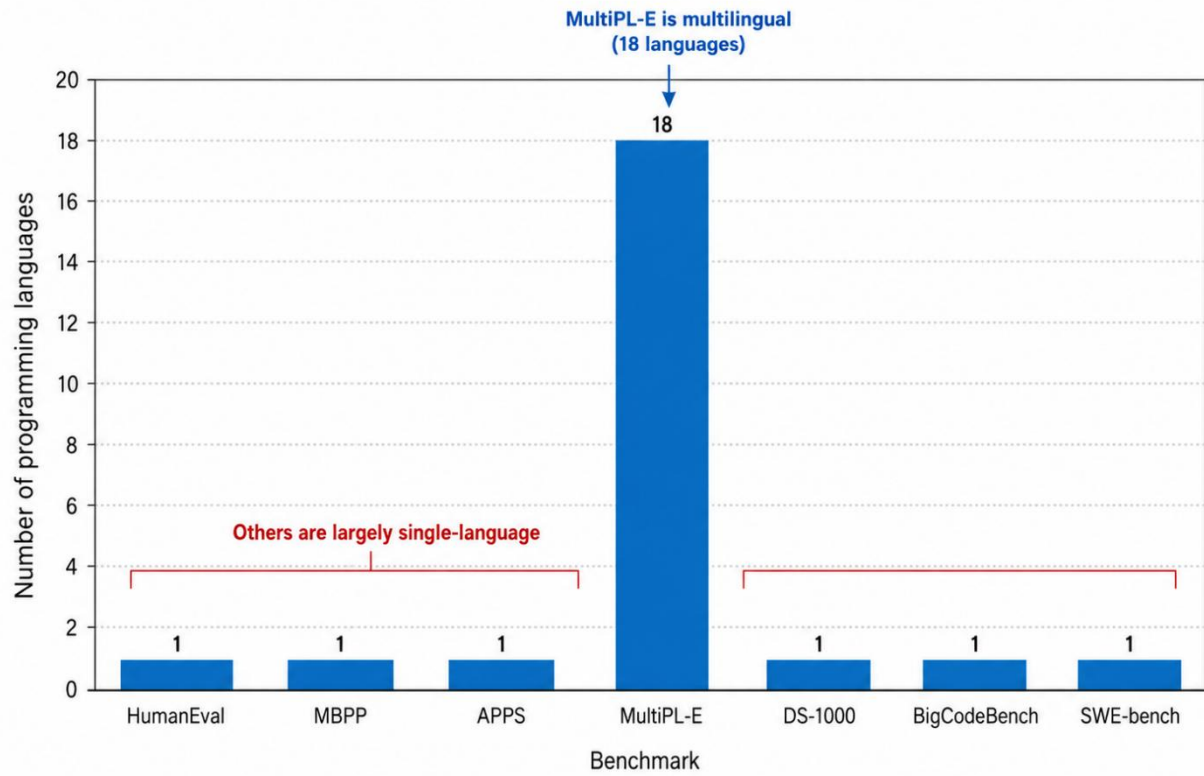
For manuscript-quality plotting, two practical notes are advisable. First, **MultiPL-E should be plotted separately by language count rather than by task count**, because its contribution is translation and breadth rather than a single canonical task count. Second, **SWE-bench and SWE-bench Verified should not be plotted as simple continuations of function-level benchmarks on the same interpretive axis**, because their unit of work is repository maintenance rather than isolated synthesis. Those are substantive scientific distinctions, not merely visual-design choices. [33]

The natural dataset portfolio is: openai/human-eval, mbpp, evalplus, bigcode/bigcodebench, xlang-ai/DS-1000, and princeton-nlp/SWE-bench or SWE-bench_Verified, using official harnesses and containerised execution where provided. The official HumanEval harness warns against executing untrusted code outside a robust security sandbox, and the official SWE-bench documentation explicitly recommends Docker-based workflows for reproducible evaluation. [21]



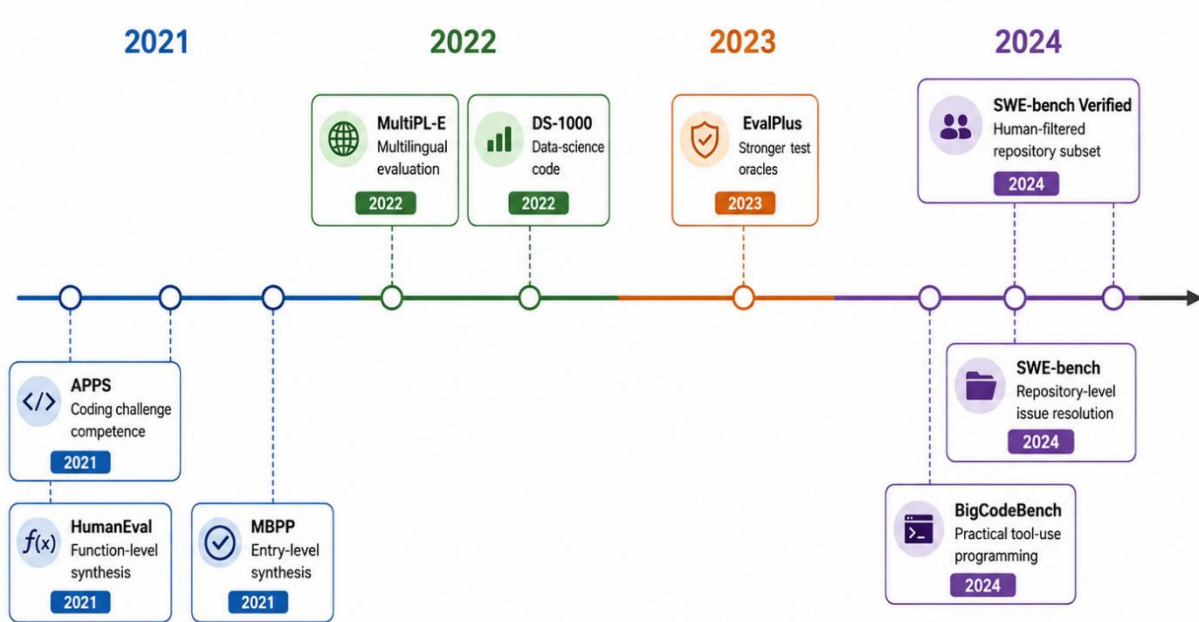
Logarithmic scaling is used because benchmark size varies substantially, from 164 HumanEval tasks to 10,000 APPS problems.

Figure 2. Benchmark Size Comparison Across Code-Generation Evaluation Sets



MultiPL-E is a multilingual benchmark family covering 18 programming languages.

Figure 3. Language Breadth of Selected Code-Generation Benchmarks



Code-generation evaluation has evolved from function-level and problem-level tasks (2021) to broader coverage with multilingual and data-science benchmarks (2022), then to stronger test oracles (2023), and finally to practical tool use and repository-level software engineering (2024).

Figure 4. Evolution of Code-Generation Benchmarks from Functions to Repositories

Declarations and Acknowledgements

Author contributions

SAMUELSON G conceived the study, defined the review scope, performed the analytical synthesis, developed the reproducibility protocol, and wrote the manuscript.

Funding

No external funding was received for this work.

Conflicts of interest

The author declares no conflicts of interest.

Ethics statement

This manuscript is based exclusively on publicly available papers, benchmark documentation, and official repositories. It does not involve human participants, animal subjects, private records, or personally identifiable data.

Reproducibility statement

This article is a structured review and protocol paper. It does not claim newly executed benchmark experiments. All empirical claims reported as benchmark facts are attributed to the originating papers or official benchmark artefacts. Any future submission derived from fresh experiments should update the methods, results, declarations, and artefact availability statements accordingly.

Acknowledgements

The author gratefully acknowledges the researchers and maintainers who released HumanEval, MBPP, APPS, MultiPL-E, DS-1000, EvalPlus, BigCodeBench, and SWE-bench, as well as the authors of recent benchmark surveys and audit studies whose open artefacts made this synthesis possible. The author should independently verify final venue compliance, formatting, and any destination-specific disclosure requirements before submission.

References

- [1] M. Chen *et al.*, “Evaluating large language models trained on code,” *arXiv preprint* arXiv:2107.03374, 2021.
- [2] J. Austin *et al.*, “Program synthesis with large language models,” *arXiv preprint* arXiv:2108.07732, 2021.
- [3] D. Hendrycks *et al.*, “Measuring coding challenge competence with APPS,” *arXiv preprint* arXiv:2105.09938, 2021.
- [4] F. Cassano *et al.*, “MultiPL-E: A scalable and extensible approach to benchmarking neural code generation,” *arXiv preprint* arXiv:2208.08227, 2022.
- [5] Y. Lai *et al.*, “DS-1000: A natural and reliable benchmark for data science code generation,” *arXiv preprint* arXiv:2211.11501, 2022.
- [6] B. Rozière *et al.*, “Code Llama: Open foundation models for code,” *arXiv preprint* arXiv:2308.12950, 2023.
- [7] J. Liu *et al.*, “Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation,” *arXiv preprint* arXiv:2305.01210, 2023.

- [8] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?” in *Proc. Int. Conf. Learn. Representations*, 2024.
- [9] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, “A survey on large language models for code generation,” *arXiv preprint* arXiv:2406.00515, 2024.
- [10] X. Hou *et al.*, “Large language models for software engineering: A systematic literature review,” *arXiv preprint* arXiv:2308.10620, 2023.
- [11] Q. Zhang *et al.*, “A survey on large language models for software engineering,” *arXiv preprint* arXiv:2312.15223, 2023.
- [12] M. L. Siddiq, S. Dristi, J. Saha, and J. C. S. Santos, “The fault in our stars: Quality assessment of code generation benchmarks,” *arXiv preprint* arXiv:2404.10155, 2024.
- [13] T. Y. Zhuo *et al.*, “BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions,” *arXiv preprint* arXiv:2406.15877, 2024.
- [14] K. Wang *et al.*, “Software development life cycle perspective: A survey of benchmarks for code large language models and agents,” *arXiv preprint* arXiv:2505.05283, 2025.
- [15] J. Cao *et al.*, “How should we build a benchmark? Revisiting 274 code-related benchmarks for LLMs,” *arXiv preprint* arXiv:2501.10711, 2025.

[1] [3] [6] [10] [14] [15] [20] [25] [27] [29] [32] <https://github.com/bigcode-project/bigcode-evaluation-harness/blob/main/docs/README.md>

<https://github.com/bigcode-project/bigcode-evaluation-harness/blob/main/docs/README.md>

[2] [9] [13] [17] <https://arxiv.org/abs/2406.00515>

<https://arxiv.org/abs/2406.00515>

[4] [8] [16] [22] <https://github.com/evalplus/evalplus>

<https://github.com/evalplus/evalplus>

[5] <https://arxiv.org/abs/2107.03374>

<https://arxiv.org/abs/2107.03374>

[7] [11] [31] [33] <https://arxiv.org/abs/2208.08227>

<https://arxiv.org/abs/2208.08227>

[12] <https://arxiv.org/abs/2406.15877>

<https://arxiv.org/abs/2406.15877>
[18] [19] [21] <https://github.com/openai/human-eval>
<https://github.com/openai/human-eval>
[23] <https://arxiv.org/pdf/2404.10155>
<https://arxiv.org/pdf/2404.10155>
[24] <https://arxiv.org/abs/2308.12950>
<https://arxiv.org/abs/2308.12950>
[26] <https://arxiv.org/abs/2211.11501>
<https://arxiv.org/abs/2211.11501>
[28] <https://arxiv.org/abs/2501.10711>
<https://arxiv.org/abs/2501.10711>
[30] <https://github.com/princeton-nlp/SWE-bench/blob/main/README.md?plain=1>
<https://github.com/princeton-nlp/SWE-bench/blob/main/README.md?plain=1>